❶ Implement IEmailSender.

❷ Inject a logger into the constructor and assign it to a private field.

❸ Use the logger to confirm which version of the IEmailSender is being used.

Within Program.cs, amend the existing registration of the `IEmailSender` interface, so the `EmailService` is only registered if the environment is `Development`, and the new `ProductionEmailService` is used for all other environments, as in the following listing.

Listing 14.6 Conditionally registering implementations based on environment

```
if (builder.Environment.IsDevelopment())      ❶
{
    builder.Services.AddTransient<IEmailSender, EmailService>();
}
else
{
    builder.Services.AddTransient<IEmailSender, ProductionEmailService>();
}
```

❶ Use the IsDevelopment extension method to determine if the environment is Development.

The `IHostEnvironment` type, exposed by the `Environment` property of both the `WebApplicationBuilder` and the `WebApplication` types, has extension methods for all the built-in environments. In addition to `IsDevelopment`, there are `IsStaging` and `IsProduction` methods. If you are using a custom environment name, you can check the value of the `Environment.EnvironmentName` to perform the same test:

```
if (builder.Environment.EnvironmentName == "QA")
```